Week 1: Introduction to DSA and Algorithm Analysis

Data Structures and Algorithms • Comprehensive Lecture Notes

1. Introduction to Data Structures and Algorithms

Computers process large volumes of data every second. To manage, store, and manipulate this information efficiently, we rely on structural designs and systematic step-by-step logic. This foundational framework is known as **Data Structures and Algorithms (DSA)**.

Definitions:

- **Data Structure (DS):** A specialized format for organizing, processing, retrieving, and storing data in a computer's memory so it can be accessed and modified efficiently. Examples include Arrays, Stacks, Queues, Trees, and Graphs.
- **Algorithm:** A finite set of well-defined instructions or step-by-step procedures designed to solve a specific problem or perform a computation.

An algorithm acts as the logic engine, while the data structure provides the organized storage container. Together, they dictate the performance and scalability of software systems.

2. Characteristics of a Good Algorithm

For an algorithm to be considered valid and effective, it must satisfy five core criteria:

1. **Input:** Zero or more quantities are externally supplied.
2. **Output:** At least one quantity is produced corresponding to the input.
3. **Definiteness:** Each instruction must be clear, unambiguous, and precise.
4. **Finiteness:** The algorithm must terminate after a finite number of execution steps.
5. **Effectiveness:** Every instruction must be basic enough to be carried out, in principle, by a person using only pencil and paper.

3. Complexity Analysis: Time and Space

When multiple algorithms solve the same problem, how do we determine which is superior? We evaluate them using resource consumption metrics known as computational complexity.

- **Time Complexity:** Measures the amount of computer time required to run an algorithm as a function of the input size (N).

- **Space Complexity:** Measures the amount of temporary memory space (RAM) required by an algorithm to execute to completion relative to input size (N).

4. Asymptotic Notations

Asymptotic analysis evaluates the performance of algorithms independent of machine hardware quirks, focusing on how execution time grows as input sizes (N) approach infinity.

Key Asymptotic Notations:

- **Big-O Notation (O):** Represents the upper bound (worst-case scenario). It describes the maximum growth rate or longest time an algorithm could possibly take.
- **Omega Notation (Ω):** Represents the lower bound (best-case scenario). It describes the minimum time required under optimal conditions.
- **Theta Notation (Θ):** Represents the tight bound (average-case or precise growth rate). It bounds the function from both above and below.

5. Common Big-O Growth Orders

Algorithms scale differently based on their internal loops and structures. Below are the most common complexity classes ranked from fastest to slowest execution scaling:

Notation	Name	Example Scenario
$O(1)$	Constant	Accessing an array element by index
$O(\log N)$	Logarithmic	Binary search in a sorted array
$O(N)$	Linear	Linear search / scanning an unsorted array
$O(N \log N)$	Linearithmic	Efficient sorting algorithms (Merge Sort, Quick Sort)
$O(N^2)$	Quadratic	Nested loops (Bubble Sort, Selection Sort)
$O(2^N)$	Exponential	Recursive Fibonacci calculation without memoization

6. Calculating Complexity Example

Consider the following simple code snippet that calculates the sum of elements in a single loop:

```
int sumArray(int arr[], int n) {
    int total = 0;              // O(1) assignment
    for(int i = 0; i < n; i++) { // Executes n times
        total += arr[i];        // O(1) per iteration
    }
    return total;               // O(1) return
}
```

Analysis: The initialization and return take constant time $O(1)$. The loop runs exactly N times, performing a constant operation inside. Therefore, the total time complexity is $T(N) = c_1N + c_2$, which simplifies to $O(N)$ in asymptotic analysis. The space complexity is $O(1)$ because storage scales independently of N.